
IoT FIRE EXTINGUISHING ROBOT

A project report submitted in partial fulfilment of
the requirements for the award of the

BACHELOR DEGREE

In

Computer Applications

from

University of Calicut



Submitted By

FAHIZ (GVXBCA009)

Carried out at



Department of Computer Applications

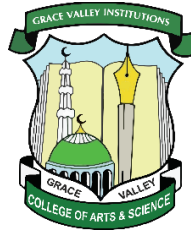
Grace Valley College of Arts & Science

Maravattam March 2026

Department of Computer Applications

Grace Valley College of Arts & Science

Maravattam



Certificate

*This is to certify that the project report entitled “IoT FIRE EXTINGUISHING ROBOT” is a bonafide record of the work done by **FAHIZ (GVAXBCA009)** under our supervision and guidance. The report has been submitted in partial fulfilment of the requirement for award of the Bachelor Degree in Computer Applications from the University of Calicut for the year 2025 -26.*

Seetha Lakshmi R
Head of Department

Varna K
Project Guide

Date:

Place:

Certified that the candidate was examined by us in the Project Viva Voice Examination held on and his/her Register Number is

External Examiner:

Declaration

I hereby declare that the project report entitled “***IoT FIRE EXTINGUISHING ROBOT***” was carried out by me as the Bachelor Degree Project in Computer Applications under the guidance and supervision of **Mrs. Seetha Lakshmi R Head & Ms. Varna K** Assistant Professor, Department of Computer Applications, Grace Valley College of Arts & Science and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

Place:

Name: FAHIZ

Date:

Acknowledgment

I am obediently thankful to God Almighty, praise and glory be to him, for all his uncountable bounties and guidance, without which, this work would have never been a reality.

An endeavor over a long period may be successful only with advice and guidance of many well wishers. I take this opportunity to express our gratitude to all who encouraged me to complete this project. I would like to express our deep sense of gratitude to my respected **Principal Dr. M USMAN**, for his inspiration and for creating an atmosphere in the college to do the project.

I would like to thank **Mrs. Seetha Lakshmi R**, Head of the department, Computer Applications for providing permission and facilities to conduct the project in a systematic way.

My sincere thanks to **Mrs. Seetha Lakshmi R, Ms. Varna K** Assistant Professors in Computer Applications for guiding and giving timely advices, suggestions and whole hearted moral support in the successful completion of this project.

Last but not least, I would like to thank all the teaching and non-teaching staff and our friends who have helped me in every possible way in the completion of my project.

Abstract

The IoT-Enabled Fire Extinguishing Robot is designed to autonomously detect and suppress small fires while providing real-time remote monitoring using the versatile ESP32 microcontroller. Unlike traditional systems that require separate Wi-Fi modules, this project leverages the ESP32's integrated wireless capabilities to combine robotic control with IoT connectivity in a single, cost-effective platform.

The system continuously monitors its environment using IR flame sensors processed by the ESP32. Upon detecting a fire, the robot executes a dual-response mechanism: physically, it navigates toward the heat source and activates a water pump to extinguish the flames; digitally, **To facilitate efficient system management, the project features a dedicated Python-based user interface complete with secure login modules, allowing stakeholders to easily track the robot's status, and manage system settings.** This ensures that property owners are immediately notified of the hazard, even if they are off-site.

This project demonstrates the efficiency of using the ESP32 for high-performance embedded applications. By integrating autonomous fire suppression with smart connectivity and a structured web interface, the robot provides a comprehensive safety solution that minimizes response time and enhances protection for homes, laboratories, and industrial environments.

Contents

	Acknowledgements	04
	Abstract	05
	Contents	06
	List of Figures	10
	List of Tables	10
	List of Abbreviations	11
1.	INTRODUCTION	12
1.1.	Introduction	12
2.	LITERATURE SURVEY	14
2.1.	Introduction	14
2.2.	Multiple Sensors Based Fire Extinguisher Robot Based on DTMF, Bluetooth and GSM Technology with Multiple Mode of Operation	17
2.2.1.	Introduction	17
2.2.2.	Methodology	17
2.3.	Fire fighter Robot: Autonomous Fire Detection and Suppression System	19
2.3.1.	Introduction	19
2.3.2.	Methodology	19
2.4.	AI in Fire Detection and Prevention: A Comprehensive Exploration	20
2.4.1.	Introduction	20
2.4.2.	Methodology	20
2.5.	Fire Fighting Robot Using Internet of Things (IoT) for Various Temperature Levels	21
2.5.1.	Introduction	21
2.5.2.	Methodology	21

2.6.	A Smart Firefighting Robot System (LAHEEB)	22
2.6.1.	Introduction	22
2.6.2.	Methodology	22
2.7.	IoT Technology Based Fire-Fighter Robot	23
2.7.1.	Introduction	23
2.7.2.	Methodology	23
2.8.	Design and Implementation of an Autonomous Fire-Extinguishing Robot with SMS Alert Functionality	24
2.8.1.	Introduction	24
2.8.2.	Methodology	24
2.9.	IoT Enabled Fire Fighting Robot Integrated With Thingspeak Cloud For Emergency	25
2.9.1.	Introduction	25
2.9.2.	Methodology	25
3.	OBJECTIVE	26
3.1.	Problem Definition	26
3.2.	Disadvantage Of Existing System	26
4.	PROPOSED SYSTEM	28
4.1.	Introduction	28
4.2.	Methodology	29
4.2.1.	System Design	30
4.2.1.1.	Data Flow Diagram	30
4.2.1.2.	Block Diagram	35
4.2.1.2.1.	Symbols Used In Block Diagrams	35
4.2.2.	Hardware Requirement Specification	37

4.2.3.	Software Requirement Specification	37
4.2.3.1.	Python	38
4.2.3.2.	HTML	41
4.2.3.3.	Visual Studio Code	41
4.2.3.3.1.	Features	42
4.2.3.4.	ESP32 Microcontroller	43
4.2.3.4.1.	Features	43
4.2.3.5.	Arduino IDE	44
4.2.3.6.	Functional Requirements	45
4.2.3.6.1.	ESP32 Microcontroller	45
4.2.3.6.2.	IR Flame Sensor	45
4.2.3.6.3.	DC Water Pump & Relay	46
4.2.4.	Experimental Setup	47
4.2.5.	Tables Used	48
5.	TESTING	52
5.1.	Unit Testing	52
5.2.	System Testing	52
5.3.	Module Level Testing	53
5.4.	Integration Testing	53
5.5.	Validation Testing	54
5.6.	Recovery Testing	54
5.7.	Output Testing	54
5.8.	Source Code Testing	55
6.	CONCLUSION	56

6.1.	Conclusion	56
6.2.	Future works	56
7.	REFERENCES	57
7.1.	Journals	57
7.2.	Books	57
7.3.	Websites	57
8.	APPENDICES	58
8.1.	Screenshots	58
9.	SOURCE CODE	61
9.1.	Hardware code	61
9.2.	Models.py	73
9.3.	Views.py	73

LIST OF FIGURES

Figure 2.1. Top view of the developed fire extinguisher robot	18
Figure 2.2. Hardware setup of Fire Fighting Robot	22
Figure 4.1. Block diagram	34
Figure 4.2. User diagram	34
Figure 4.3. Admin diagram	34
Figure 4.4. Arduino IDE	44
Figure 4.5. ESP32 Microcontroller	45
Figure 4.6. IR Flame Sensor	46
Figure 4.7. Relay	46
Figure 4.8. DC Water Pump	47
Figure 4.9. IoT Fire Extinguishing Robot.....	47

LIST OF TABLES

Table 1: Register	51
-------------------------	----

LIST OF ABBREVIATIONS

1. **ADC** = ANALOG-TO-DIGITAL CONVERTER
2. **AI** = ARTIFICIAL INTELLIGENCE
3. **CSS** = CASCADING STYLE SHEETS
4. **DC** = DIRECT CURRENT
5. **DFD** = DATA FLOW DIAGRAM
6. **DTMF** = DUAL-TONE MULTI-FREQUENCY
7. **ERD** = ENTITY RELATIONSHIP DIAGRAM
8. **GSM** = GLOBAL SYSTEM FOR MOBILE COMMUNICATIONS
9. **HTML** = HYPERTEXT MARKUP LANGUAGE
10. **HTTP** = HYPERTEXT TRANSFER PROTOCOL
11. **I2C** = INTER-INTEGRATED CIRCUIT
12. **IDE** = INTEGRATED DEVELOPMENT ENVIRONMENT
13. **IoT** = INTERNET OF THINGS
14. **IR** = INFRARED
15. **OS** = OPERATING SYSTEM
16. **PWM** = PULSE WIDTH MODULATION
17. **SPI** = SERIAL PERIPHERAL INTERFACE
18. **SQL** = STRUCTURED QUERY LANGUAGE
19. **UI** = USER INTERFACE
20. **Wi-Fi** = WIRELESS FIDELITY

1. INTRODUCTION

1.1 INTRODUCTION

In our rapidly evolving world, the imperative to ensure safety in our homes and industries has never been more pressing. With the continual emergence of new technologies within the realm of the Internet of Things (IoT) and robotics, we stand at the threshold of a revolutionary shift in how we approach disaster management and emergency response. At the forefront of this transformative movement is our pioneering project: the IoT-Enabled Fire Extinguishing Robot, meticulously designed to cater specifically to the unique demands of modern fire safety.

Fire safety systems serve as indispensable guardians of our living and working spaces, protecting lives and property from the devastating effects of uncontrolled flames. Yet, within these environments, the threat of fire outbreaks looms ominously, posing significant risks to occupants and valuable assets alike. Traditional methods often rely on human intervention or passive alarms, which can be slow to react. It is this critical juncture where our project steps in, presenting a solution that not only elevates safety standards but also introduces autonomous intervention, ultimately fostering a safer environment for all.

Envision a system that transcends conventional smoke detectors and manual extinguishers. Our IoT-based Fire Extinguishing Robot does not merely exist; it actively surveils the environment, casting a watchful eye over the area through an intricate network of IR flame sensors and instantaneous data analysis using the ESP32 microcontroller. It remains ever-vigilant, tirelessly scanning for any indications of potential fire hazards, poised to act at a moment's notice without risking human life.

Yet, the system's functionality extends far beyond mere detection. In the unfortunate event of a fire, time becomes an invaluable asset. Our system springs into action with unwavering speed and precision. Physically, it navigates directly toward the heat source to extinguish the flames using its onboard water pump. Simultaneously, This secure interface features distinct

Admin and User login modules, allowing stakeholders to monitor the robot's real-time status, manage system access, and review incident logs effortlessly. This rapid dual-response mechanism significantly reduces the time it takes to control the situation, potentially turning the tide in critical scenarios and preventing minor accidents from becoming major disasters.

Indeed, our IoT Fire Extinguishing Robot stands as a transformative force within the realm of safety automation. It represents a paradigm shift, offering property owners the reassurance of top-notch, autonomous protection and real-time digital oversight even when they are miles away. For industrial workers and residents, it instills a newfound sense of confidence, ensuring that their environment is safeguarded by cutting-edge robotic measures. Moreover, for our communities at large, it marks a substantial stride in harnessing technology to protect every individual from the dangers of fire.

In conclusion, our IoT Fire Extinguishing Robot serves as a beacon of innovation in the realm of disaster prevention. It showcases the immense potential of technology when coupled with an unyielding commitment to safety. As we embark on this transformative journey, we demonstrate how robotics, IoT connectivity, and modern software design can converge seamlessly to safeguard lives and foster a community of secure, intelligent protection

2. LITERATURE SURVEY

2.1 INTRODUCTION

Fire Safety and the Need for Automated Solutions

Fire safety has long been a major concern in industrial, residential, and public domains due to the potential for catastrophic loss of life and property. Fires cause extensive damage to infrastructure and the environment, making effective detection and prevention mechanisms imperative. Despite safety regulations, fire incidents continue to pose significant threats, highlighting the urgent need for technological advancements that can improve emergency response times and minimize human risk.

Challenges in Traditional Firefighting

Traditional systems such as smoke alarms and sprinklers provide only partial solutions and often depend on human intervention to be truly effective. Conventional firefighting techniques expose human responders to dangerous environments, often lagging in both efficiency and personal protection. Furthermore, reliance on human notification can lead to delays in alerting fire services, allowing small fires to escalate into uncontrollable hazards before help arrives.

The Potential of IoT-based Firefighting Systems

The emergence of the Internet of Things (IoT) and robotics presents a promising opportunity to revolutionize fire safety mechanisms. By integrating autonomous navigation with real-time data transmission, IoT-based systems can detect accidents and automatically trigger, significantly reducing response times. These systems leverage sensors and microcontrollers to monitor environments continuously, offering a proactive approach to disaster management that traditional passive alarms cannot match.

Existing Research on Firefighting Robots

A growing body of research explores the development of firefighting robots capable of autonomous navigation, detection, and suppression. Early iterations often relied on manual control via DTMF or Bluetooth, requiring a human operator to be within range. More advanced systems utilize multiple sensors—such as flame, smoke, and temperature sensors—to ensure accurate detection.

Existing research also explores various communication protocols for remote alerting. While some systems use GSM modules to send SMS alerts, others integrate Wi-Fi to send data to cloud platforms like ThingSpeak for real-time monitoring. However, many of these solutions suffer from high complexity and cost due to the need for separate microcontrollers (like Arduino) and external communication modules.

Focus of this Project: IoT-Enabled Fire Extinguishing Robot using ESP32

Integrated Connectivity: Unlike traditional systems that require separate Wi-Fi or GSM modules, this project leverages This project builds upon existing research by proposing a compact, cost-effective IoT-Enabled Fire Extinguishing Robot driven by the ESP32 microcontroller. This focus presents unique advantages:

- the ESP32's integrated wireless capabilities to combine robotic control with IoT connectivity in a single platform.
- **Mechanism:** The system is designed to perform two critical tasks simultaneously: physically navigating toward the heat source to extinguish flames using a pump

Literature Review and Contribution to Existing Knowledge

This project aims to contribute to the existing body of knowledge on autonomous safety systems by:

- **Demonstrating the efficiency of the ESP32:** This project will investigate the effectiveness of using the ESP32 for high-performance embedded applications, proving that complex safety solutions can be built on cost-effective hardware.
- **Integrating autonomous suppression with smart connectivity:** The study will explore how combining immediate physical suppression with remote alerting provides a comprehensive safety solution that minimizes response time for homes and laboratories.
- **Enhancing accessibility to safety technology:** By reducing hardware complexity and cost, this project aims to make advanced fire safety robotics more accessible for varied environments, from small workshops to residential buildings.

2.2 Multiple Sensors Based Fire Extinguisher Robot Based on DTMF, Bluetooth and GSM Technology with Multiple Mode of Operation

2.2.1 INTRODUCTION

Fire accidents are a major catastrophe that can cause massive loss of life and property. Traditional fire extinguishing systems often rely on manual operation or simple smoke alarms, which may not be sufficient in hazardous or unmanned environments. To address this, the proposed system introduces a multi-functional robot capable of detecting and extinguishing fires autonomously or via remote control.

The primary objective of this project is to design a robot that ensures high reliability in fire detection by combining three different types of sensors: flame, temperature, and smoke sensors. Unlike single-sensor systems, this multi-sensor approach reduces false alarms. Furthermore, the robot integrates **DTMF (Dual-Tone Multi-Frequency)**, **Bluetooth**, and **GSM** technologies, allowing users to control the robot from long distances and receive immediate SMS alerts with location coordinates when a fire is detected.

2.2.2 METHODOLOGY

The system is built around an **Arduino Mega** microcontroller, which serves as the central processing unit to manage sensor inputs and control the robot's movement. The robot is designed to operate in three distinct modes to ensure versatility in different scenarios. In the **Autonomous Mode**, the robot continuously monitors the environment using its sensor array; if flame, smoke, or temperature levels exceed a set threshold, it automatically navigates toward the source and activates the water pump. Alternatively, the **Line Following Mode** allows the robot to patrol specific paths by tracking a black line on the floor, making it suitable for routine surveillance in corridors. Finally, the **Manual Control Mode** gives the user complete control over the robot via **DTMF** tones for long-range operation over a GSM network, or via **Bluetooth** using an Android smartphone for short-range responsiveness.

Throughout all modes, if a fire is detected, the integrated GSM module transmits an SMS alert containing GPS coordinates to the user and fire department, ensuring immediate remote notification.



Fig.2.1 Top view of the developed fire extinguisher robot.

2.3 Fire fighter Robot: Autonomous Fire Detection and Suppression System

2.3.1 INTRODUCTION

Fire incidents reflect a significant threat to human life and property, and conventional firefighting techniques often expose employees to dangerous environments while lagging in both efficiency and protection. To address this, firefighter robots provide a technological solution by ensuring a speedy reaction and decreasing human involvement in hazardous zones. This paper offers a comprehensive evaluation of firefighter robots, monitoring their structure, components, and underlying technologies. The study highlights the potential of robotic firefighting structures to enhance emergency response efficiency, thereby improving the safety of humans and property in critical situations.

2.3.2 METHODOLOGY

The system is designed with a robust multi-sensor architecture that includes flame sensors to detect fire-prone areas and infrared sensors to capture heat radiation for accurate identification. For navigation, the robot employs wheels or tracks capable of obstacle avoidance, utilizing LIDAR-based technology for autonomous pathfinding to reach the target location safely. A key feature of this methodology is the dual-mode fire suppression system, which can deploy water jets for standard fires or dry chemical extinguishers for more critical scenarios. The entire operation is supported by advanced technologies, including AI for autonomous decision-making and IoT for cloud-based remote monitoring, ensuring real-time data transmission to human operators.

2.4 AI in Fire Detection and Prevention: A Comprehensive Exploration

2.4.1 INTRODUCTION

Artificial Intelligence (AI) has transformed various industries by enhancing efficiency and precision, and its applications in fire detection and prevention are increasingly gaining prominence. Fires cause extensive damage to life, property, and the environment, making effective detection and prevention mechanisms imperative. This research delves into the integration of AI in fire management systems, focusing on advanced technologies such as computer vision, machine learning, and neural networks. It examines AI's capabilities in early fire detection, real-time monitoring, predictive analysis, and risk assessment, underscoring its potential to surpass traditional approaches in speed and accuracy.

2.4.2 METHODOLOGY

The proposed system utilizes AI algorithms, specifically Convolutional Neural Networks (CNN) and machine learning models, to detect fire and smoke from images and video feeds. A Computer Vision system provides real-time flame and smoke recognition, significantly reducing human error in detection. Additionally, IoT sensors are integrated to collect environmental data such as temperature and gases, which feeds into the system for broader analysis. Predictive analytics are then employed to anticipate fire spread, particularly in outdoor environments, allowing for proactive rather than just reactive measures. This combination of visual data processing and environmental sensing creates robust, automated detection mechanism.

2.5 Fire Fighting Robot Using Internet of Things (IoT) for Various Temperature Levels

2.5.1 INTRODUCTION

The integration of Internet of Things (IoT) technology with fire-fighting systems has emerged as a promising solution for enhancing the efficiency and effectiveness of emergency responses. This paper presents a comprehensive system that leverages IoT devices to detect and respond to fire incidents autonomously. The primary goal is to minimize potential damage and risks to human life, particularly in large-scale industries like chemical factories and petroleum refineries where catastrophic outcomes are common. By utilizing a cloud-based platform, the system ensures that fire data is not just detected but also analyzed for further investigation.

2.5.2 METHODOLOGY

The proposed system is built upon an **Arduino UNO** microcontroller which acts as the central control unit. It is equipped with a comprehensive sensor array including **Flame, Smoke, and Temperature sensors** to detect fire outbreaks in real-time. Upon detection, the robot utilizes **DC motors** driven by an L298N motor driver to navigate towards the fire source and employs a **water pump with a nozzle** to extinguish the flames. A defining feature of this methodology is the integration with the **ThingSpeak cloud platform**; all sensor data is transmitted to the cloud server via Wi-Fi for real-time monitoring and analysis. The robot is designed to operate in two modes: an **autonomous control system** for independent reaction and a **manual control system** for human-guided intervention. Additionally, a GSM module is incorporated to send immediate alerts to users or fire services, ensuring a swift response.

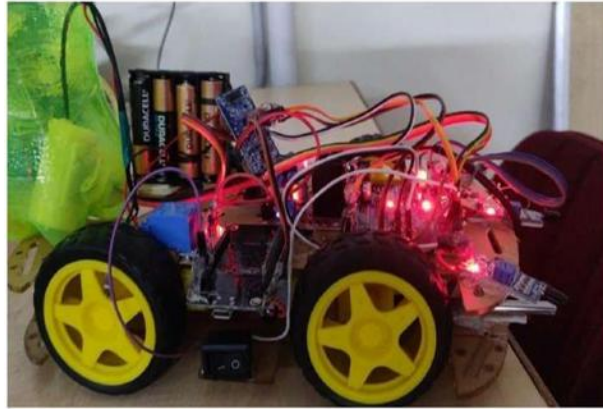


Fig.2.2 Hardware setup of Fire Fighting Robot

2.6 A Smart Firefighting Robot System (LAHEEB)

2.6.1 INTRODUCTION

The motivation behind this project stems from the inherent dangers faced by firefighters, whose duties often lead to injury or loss of life. To mitigate these risks, this paper proposes a smart firefighting robot named "LAHEEB," designed not only to detect and extinguish fires but also to increase knowledge about fire behavior in incident areas. The system is capable of extinguishing various classes of fire—including types A, B, C, D, F/K, electrical, and metal fires—in the shortest possible time, thereby reducing monetary losses and protecting human responders from hazardous environments.

2.6.2 METHODOLOGY

The robot functions as a comprehensive autonomous unit powered by an **Arduino UNO (ATmega328P)** microcontroller, which serves as the system's brain. It features a sophisticated sensor array, including a **Five-Channel Flame Sensor** that provides a wide detection angle of over 120 degrees, and an **Ultrasonic Sensor (HC-SR04)** mounted on a servo motor for precise obstacle detection and navigation. To ensure accurate fire confirmation, a **Temperature Sensor (LM335A)** monitors environmental heat. For suppression, the robot utilizes a liquid tank and a spray mechanism; the spraying nozzle is mounted on a servo motor to cover a maximum area, while

a **12V pump** propels the extinguishing agent. The entire system is housed in a compact, heat-resistant **aluminum alloy chassis**, making it durable and suitable for entering small or dangerous spaces.

2.7 IoT Technology Based Fire-Fighter Robot

2.7.1 INTRODUCTION

Detecting and extinguishing fires is inherently a hazardous job for human personnel, often exposing them to life-threatening risks. This project introduces an **IoT firefighter robot** as a solution to perform these tasks efficiently while ensuring safety. The system is designed to be both autonomous and manually controlled, providing flexibility in different emergency scenarios. A distinguishing feature of this project is the use of a **Remote Desktop application** for control, where the robot and the controlling authority are connected via the internet. This allows for precise identification of fire types, enabling the operator to apply the appropriate extinguishing methods remotely.

2.7.2 METHODOLOGY

The robot's control unit is based on the **Arduino UNO microcontroller**, which processes data from a suite of environmental sensors including **Flame sensors**, **Temperature sensors (LM35/DHT11)**, and **Smoke sensors (MQ-2)**. For mobility, it utilizes **DC motors** driven by an **L298N motor driver**, while an **Ultrasonic sensor** ensures obstacle avoidance during navigation. The fire extinguishing mechanism consists of a water pump connected to a tank, with a **servo motor** controlling the nozzle direction to target the flames accurately. A critical component of the methodology is the integration of **live video streaming via a webcam**, which allows the "authority" (user) to monitor the environment in real-time through the desktop application and make informed decisions on whether to switch between autonomous and manual modes.

2.8 Design and Implementation of an Autonomous Fire-Extinguishing Robot with SMS Alert Functionality

2.8.1 INTRODUCTION

Fire outbreaks represent a formidable threat capable of causing significant loss of life, property destruction, and permanent injuries. While firefighters bravely confront these hazards, they often place themselves at serious risk to protect communities. This study addresses these dangers by designing a firefighting robot that utilizes machine learning and cutting-edge sensor technologies. The primary objective is to enhance fire detection accuracy and minimize false alarms, thereby reducing reliance on human intervention and enhancing safety for both firefighters and civilians

2.8.2 METHODOLOGY

The robot is developed using the **C++ language** on an **Arduino UNO microcontroller** (based on the ATmega32), which controls the entire system. For navigation, it employs **ultrasonic sensors** and a reinforcement learning approach to avoid obstacles and move through rooms with high precision. Fire detection is handled by a combination of **flame sensors** and a **smoke detector (MQ-2)**, ensuring swift identification of hazards. A unique feature of this methodology is the **SMS capability**; upon detecting a fire, the robot not only activates a water pump to suppress the flames autonomously but also sends an immediate SMS notification to the concerned entity, ensuring that human authorities are alerted even if they are off-site.⁶⁹

2.9 IoT Enabled Fire Fighting Robot Integrated With Thingspeak Cloud For Emergency

2.9.1 INTRODUCTION

The integration of Internet of Things (IoT) technology with fire-fighting systems has emerged as a promising solution for enhancing the efficiency and effectiveness of fire emergency response. This paper presents a comprehensive overview of a fire-fighting robot integrated with the **ThingSpeak** platform. The proposed system leverages IoT capabilities to detect and respond to fire incidents autonomously, minimizing potential damages and risks to human life. This approach is particularly vital for large-scale businesses such as chemical factories and petroleum refineries, where fire incidents can have catastrophic outcomes.

2.9.2 METHODOLOGY

The system is built around an **Arduino UNO** microcontroller, which acts as the central control unit, managing inputs from a variety of sensors including **temperature, smoke, and gas sensors** to detect outbreaks in real-time. Upon detection, the robot utilizes its onboard actuators—specifically **DC motors** for mobility and a **water pump with a nozzle**—to contain and extinguish the fire. The robot is capable of operating in both an **autonomous control system** and a **manual control system**, offering flexibility in different scenarios. A key feature of this methodology is the communication system; all sensor data is sent to the **ThingSpeak cloud server** via Wi-Fi for further investigation and real-time monitoring.

3. OBJECTIVE

3.1 PROBLEM DEFINITION

We have taken eight papers for the review to analyze the current state of firefighting robotics and identify the gaps in existing technology.

3.2 DISADVANTAGE OF EXISTING SYSTEM

1. Dependency on Human Intervention: Many existing firefighting robots rely heavily on manual control mechanisms, such as Bluetooth or DTMF remotes, which require a human operator to be within a specific range to direct the robot. This dependency delays the response time and exposes the human operator to potential danger. Inaccurate Location Information: Without advanced GPS technology, existing systems might struggle to provide precise location data for the accident. This can result in emergency services having difficulty locating the scene, especially if the accident is in an unfamiliar area.

2. High Cost and Complexity: A significant number of reviewed systems utilize complex hardware configurations, often combining an Arduino microcontroller with separate external modules for GSM, GPS, and Wi-Fi. This not only increases the overall cost of the device but also adds to the circuit complexity and power consumption, making it less accessible for general use.

3. Delayed Response: In traditional setups, there is often a lag between fire detection and the actual suppression effort, particularly if the system waits for human confirmation or manual activation. This delay can be critical, allowing small fires to escalate into uncontrollable hazards before the robot engages.

4. Limited Range of Operation: Systems that rely on Bluetooth or RF remotes are restricted by short operational distances. If the fire occurs in a large

facility or a remote corner of a building, the operator may not be able to control the robot effectively from a safe distance.

5. False Alarms: Some existing solutions rely on single-sensor detection (e.g., just a smoke sensor), which can lead to false positives triggered by non-hazardous smoke or dust. This reduces the reliability of the system and causes unnecessary panic.

4. PROPOSED SYSTEM

4.1 INTRODUCTION

In the realm of fire safety and disaster management, the IoT-Enabled Fire Extinguishing Robot represents a groundbreaking innovation poised to revolutionize how we address and respond to fire outbreaks. Developed as a collaborative effort for a college final year project, this system aims to provide a swift and automated response mechanism in the critical moments following the ignition of a fire.

The motivation behind this system stems from the urgent need for more efficient and proactive measures to handle fire accidents, particularly those in hazardous environments where human intervention is risky. Often, our team embarked on creating a solution that not only detects fires promptly but also ensures that suppression efforts begin immediately with unprecedented speed and accuracy.

At its core, the Fire Extinguishing Robot is a fusion of cutting-edge technologies seamlessly integrated to function as a cohesive unit. Central to its operation is the utilization of Internet of Things (IoT) principles, where onboard sensors specifically designed to detect flame wavelengths are employed to identify the occurrence of a fire. These sensors are finely tuned to recognize infrared radiation indicative of flames, immediately triggering the system into action.

Upon fire detection, the system springs into action, utilizing its autonomous navigation capabilities to move toward the heat source. This real-time sensor data serves as the cornerstone for the subsequent steps in the emergency response process. Simultaneously, This secure portal features distinct login modules, ensuring that authorized personnel are instantly informed, can securely manage the system remotely, and review comprehensive incident logs. In essence, the IoT Fire Extinguishing Robot is not merely a project but a mission to enhance safety and emergency response efficacy. By leveraging the power of the ESP32 microcontroller, robotics, instant communication

protocols, and a structured web interface, it offers a proactive approach to mitigating the consequences of fire accidents. This system aspires to be a vital tool for safety, ensuring that hazardous situations are neutralized swiftly and efficiently, ultimately contributing to safer homes and industrial workspaces.

4.2 METHODOLOGY

The IoT Fire Extinguishing Robot's methodology encompasses a multifaceted approach integrating hardware, embedded software, and wireless communication components. The hardware setup involves equipping a robotic chassis with **IR flame sensors**, a **water pump**, and **DC motors**. These sensors, designed to identify the specific wavelength of fire, are connected to an **ESP32 microcontroller**, the system's brain.

Upon detection of a flame, the ESP32 processes the input from the sensors to determine the direction of the fire. It then triggers the motor driver to navigate the robot toward the source. Once in range, the microcontroller activates the relay to power the water pump to direct the water jet accurately onto the flames.

Simultaneously, the ESP32 utilizes its integrated Wi-Fi module. The software logic ensures that the robot continues to spray water until the sensors no longer detect the presence of fire.

The system's firmware is developed using the **Arduino IDE (C++)**, ensuring efficient handling of sensor data and motor control logic. Through this methodology, the project integrates IoT principles, autonomous robotics, and instant communication protocols to deliver a comprehensive solution for enhanced fire safety and efficient emergency response.

4.2.1 SYSTEM DESIGN

4.2.1.1 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of how data flows through a system, showing the processes, data stores, and external entities that interact with the system. It's a powerful tool used in system analysis and design to visualize and understand the flow of data within an information system.

At the core of a DFD are four main components:

1. **Processes:** These are the functions or transformations that are performed on the data. Each process takes inputs, does something with them, and produces outputs. Processes in a DFD are represented by circles or ovals. In our project, the main process is the **"Fire Detection and Control Algorithm"** running on the ESP32.
2. **Data Flows:** These represent the movement of data between processes, data stores, and external entities. Data flows are shown as arrows and indicate the direction in which data is moving. In our system, the **"Sensor Readings"** flow from the sensors to the microcontroller, and **"Control Signals"** flow from the microcontroller to the motors.
3. **Data Stores:** These are where data is held within the system. They represent databases, files, or even collections of documents where data is stored for later use. Data stores are depicted as rectangles. In our IoT implementation, the **"Cloud Database"** (e.g., ThingSpeak/Firebase) acts as a data store for logging fire incidents.
4. **External Entities:** These are sources or destinations of data outside the system being modeled. They could be users, other systems, or organizations that interact with the system. External entities are shown as squares. For this project, the **"Environment"** (source of fire) and the **"User/Smartphone"** (receiver of alerts) are the external entities.

When creating a DFD, there are different levels, from the high-level overview to more detailed levels. The most common levels are Context Diagram, Level 0, and then subsequent lower levels for more detail.

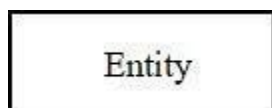
- **Context Diagram (Level 0):** This is the highest level and shows the **Fire Extinguishing Robot System** as a single process. It interacts with the **Environment** (Input: Heat/Fire) and the **User**. It provides an overall view of the system's boundaries.
- **Level 1 DFD:** This is the next level down, breaking the single process of the Context Diagram into subprocesses. It shows the main internal functions such as "**Read Sensors**," "**Process Data**," "**Drive Motors**," showing how they interact with each other.

DFDs are valuable because they provide a clear picture of how data moves through a system, aiding in understanding system requirements. They help identify potential bottlenecks or areas for improvement in a system's design and serve as a communication tool between stakeholders, ensuring everyone has a common understanding of the system's functionality.

In summary, a Data Flow Diagram is a visual representation of the flow of data within a system, using processes, data stores, data flows, and external entities. It helps in understanding, designing, and communicating the workings of complex systems.

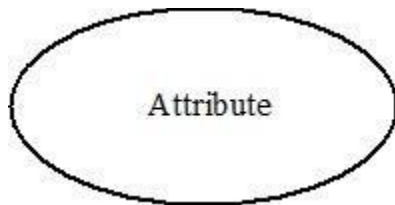
Entity:

An entity is a person, place, thing, or event of interest to the organization and about which data are captured, stored, proceed. For example, an employee is an entity.



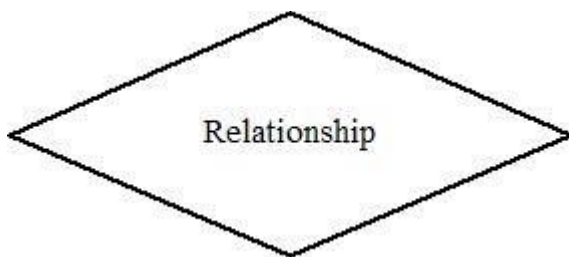
Attribute:

Various type of data item that describe an entity are known as attributes. For example name, address, DOB (Date of Birth) etc. are the attribute of the entity employee.



Relationships:

An association of several entities is an Entity-Relation mode is called relationship.



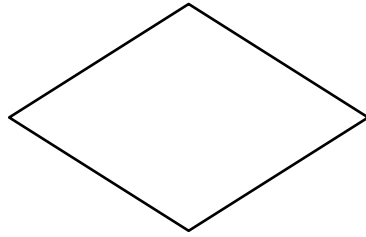
While drawing the Entity-Relationship diagram, entity names are represented by a rectangle, relationships are represented by diamond and oval shapes are used for representing attributes.

Rectangle



Represent entity set

Diamond



Represents relationships

Line



Links attributes to entity sets and entity

In diagramming techniques, entities are named and represented inside and represented inside a rectangular box. An association among entities leads to relationships. Three type of relationships exists among entities.

These are:

One-to-one (1:1)

One-to-many (1:M)

Many-to-many (M:M)

One-to-One (1:1) Relationship

In the world of database design, a one-to-one (1:1) relationship is a straightforward connection between two entities. Imagine you have two entities, such as "Person" and "Passport." Each person has exactly one passport, and each passport belongs to only one person. This is a classic example of a one-to-one relationship. Visually, in diagramming techniques

like Entity-Relationship Diagrams (ERDs), you would represent this as a line connecting the two entities.

One-to-one relationships are used when each record in one entity corresponds to exactly one record in another entity. They are useful for situations where two entities have a unique and singular connection, like a person and their social security number, or an employee and their employee ID. These relationships are relatively simple and direct, making them easy to understand and implement in database design.

One-to-Many (1:M) Relationship

Moving on to one-to-many (1:M) relationships, we enter a more common scenario in databases. This relationship describes a situation where one entity is associated with multiple instances of another entity. For example, consider the relationship between "Customer" and "Orders." A customer can place multiple orders, but each order belongs to only one customer. This is a one-to-many relationship, often depicted in ER diagrams with a line connecting the "Customer" entity to the "Orders" entity.

One-to-many relationships are fundamental in relational databases. They reflect realworld scenarios where one entity can have multiple related records in another entity. In a school database, a student can enroll in multiple courses, but each course has multiple students. This relationship allows for efficient organization and retrieval of data, enabling applications to manage diverse sets of information effectively.

Many-to-Many (M:M) Relationship

Now, let's explore the many-to-many (M:M) relationship, which represents a more complex scenario. In this type of relationship, multiple records in one entity can be associated with multiple records in another entity. A classic example is the relationship between "Students" and "Courses." Many students can enroll in many courses, and conversely, many courses can have many students enrolled. This many-to-many relationship is depicted in ER diagrams with a line connecting "Students" to "Courses."

Many-to-many relationships are common in various systems, from student-course enrollments to product categories in an e-commerce platform. Implementing this relationship in a database usually requires a linking table, also known as a junction or associative table. This table resolves the many-to-many relationship by creating pairs of foreign keys, linking records from each entity.

4.2.1.2 BLOCK DIAGRAM

A block diagram is a visual representation of a system or process using simple, geometric shapes to depict its components and their interactions. Each block represents a different part of the system, such as functions, operations, or subsystems, and arrows show the flow of information or signals between them. Block diagrams are used in engineering, electronics, and systems design to provide a high-level overview, making complex systems easier to understand. They are valuable tools for conceptualizing, analyzing, and communicating the structure and functionality of systems in a clear and concise manner.

4.2.1.2.1 SYMBOLS USED IN BLOCK DIAGRAMS

Block diagrams use very basic geometric shapes: boxes and circles. The principal parts and functions are represented by blocks connected by straight and segmented lines illustrating relationships. When block diagrams are used in electrical engineering, the arrows connecting components represent the direction of signal flow through the system. Whatever any specific block represents should be written on the inside of that block. A block diagram can also be drawn in increasing detail if analysis requires it. Feel free to add as little or as much detail as you want using more specific electrical schematic symbols.

- Identify the system. Determine the system to be illustrated. Define components, inputs, and outputs.
- Create and label the diagram. Add a symbol for each component of the system, connecting them with arrows to indicate flow. Also, label each block so that it is easily identified.

BLOCK DIAGRAM



Fig 4.1 Block diagram

USER

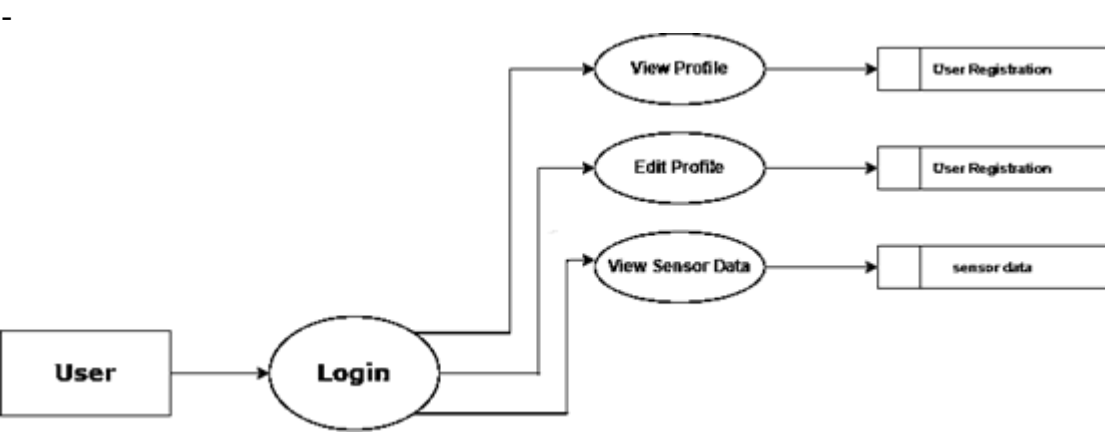


Fig. 4.2 User diagram

ADMIN

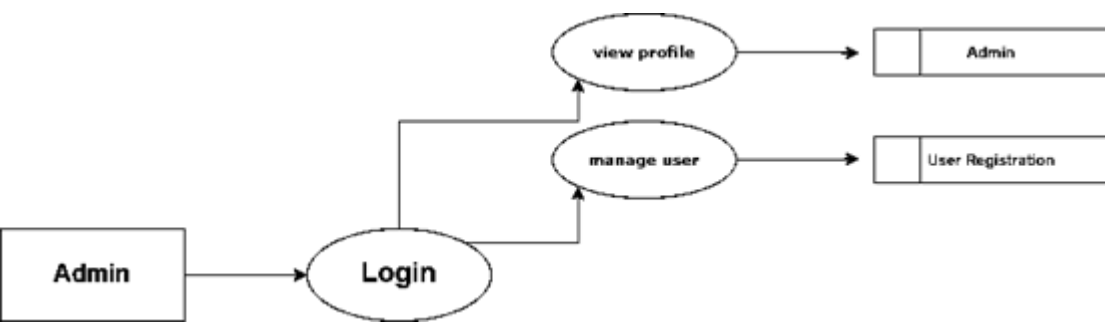


Fig.4.3 Admin diagram

4.2.2 HARDWARE REQUIREMENT SPECIFICATION

The selection of hardware is very important in the existence and proper working of any of the software. When selecting hardware, the size and capacity requirements are also important. The hardware must suit all application developments.

Processor	:	i3 or Above
System Bus	:	32 Bit or 64 Bit
RAM	:	4 GB or Above
HDD	:	500 GB or Above

4.2.3 SOFTWARE REQUIREMENT SPECIFICATION

One of the most difficult tasks is selecting software, once the system requirement is find out then we have to determine whether a particular software package fits for those system requirements. This section summarizes the application requirement.

Operating System bit platform	:	Windows 10 any 32 bit or 64
Front End	:	HTML, CSS
Back End	:	Sqlite, Arduino IDE
Framework	:	Django
IDE	:	Visual Studio Code (VS code)
Script above	:	Python 3.8 or above

4.2.3.1 PYTHON

Python is a widely used general-purpose, high level programming language. It was initially designed by Guido van Rossum in 1991 and developed by Python Software Foundation. It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code. Python is a programming language that lets you work quickly and integrate systems more efficiently. There are two major Python versions- Python 2 and Python 3. Both are quite different.

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are fully supported, and many of its features support functional programming and aspect-oriented programming (including by meta programming and meta objects (magic methods)). Many other paradigms are supported via extensions, including design by contract and logic programming. Python uses dynamic typing and a combination of reference counting and a cycle-detecting garbage collector for memory management. It also features dynamic name resolution

(late binding), which binds method and variable names during program execution.

Python's design offers some support for functional programming in the Lisp tradition. It has filter, map, and reduce functions; list comprehensions, dictionaries, sets, and generator expressions. The standard library has two modules (itertools and functools) that implement functional tools borrowed from Haskell and Standard ML. Python is meant to be an easily readable language. Its formatting is visually uncluttered, and it often uses English keywords where other languages use punctuation. Unlike many other languages, it does not use curly brackets to delimit blocks, and semicolons after statements are optional. It has fewer syntactic exceptions and special cases than C or Pascal. Python's large standard library, commonly cited as one of its greatest strengths, provides tools suited to many tasks. For Internet-facing applications, many standard formats and protocols such as MIME and HTTP are supported. It includes modules for creating graphical user

interfaces, connecting to relational databases, generating pseudorandom numbers, arithmetic with arbitrary-precision decimals, manipulating regular expressions, and unit testing. Most Python implementations (including CPython) include a read–eval–print loop (REPL), permitting them to function as a command line interpreter for which the user enters statement sequentially and receives results immediately. Other shells, including IDLE and IPython, add further abilities such as auto-completion, session state retention and syntax highlighting. As well as standard desktop integrated development environments, there are Web browser- based IDEs; Sage Math (intended for developing science and math-related Python program); Python Anywhere, a browser-based IDE and hosting environment; and Canopy IDE, a commercial Python IDE emphasizing scientific computing.

FEATURES OF PYTHON LANGUAGE ARE:

- Simple and easy to Learn:

The syntaxes of python language are very simple. Anybody can remember the python language syntaxes, rules and regulations very easily. The elegant syntaxes of the Python language make the people learn python most comfortably. Without having any other programming languages knowledge, we can learn python directly. The simple and powerful syntax of python language makes the programmers express their business logic in fewer lines of code.

- Platform Independent:

Like Java programs, Python programs are also platform independent. Once we write a Python program, it can run on any platform without rewriting once again. Python uses PVM to convert python code to machine understandable code.

- Portability:

The python applications are portable. The applications which are developed by python on one platform are going to execute on any platform without making any changes (recompiling is not required). To achieve the portability feature for every operating system a separate python software is developed for every version.

- Free and Open Source & Redistribution:

Python is an open source language so that anyone can use the python software without purchasing the licence. Anyone can read the python source code and can be able to do the modification in the source code and also, we can redistribute that code to others. •HighLevel Language: Python is a high-level language. While developing python applications, developers do not need to bother about memory management.

- Dynamically Typed Language:

Python is a dynamically typed language. I mean, in python there is no need to declare the type of a variable. Whenever we assign a value to the variable, based on the value the type will be allocated automatically.

- Python supports POP & OOP:

Python language supports both Procedural Oriented Programming and ObjectOriented Programming features. We can implement OOPs features like Encapsulation, polymorphic, inheritance and abstraction in the Python programming language.

- Python is interpreted Language:

Like PHP, Python is also an interpreted language. Python applications do not require explicit compilation so that a compiler is not required for python software. Python interpreter is responsible for the execution of python

applications. Whenever we run a python application, the python interpreter checks for the syntax errors. If there are no syntax errors found in our code, then the interpreter converts the code into intermediate code in the form of low-level format and executes it. The intermediate code of the python applications is known as byte code. The extension of the byte code file is .py (Python Compile Code).

- Python is Embeddable:
- We can embed the python code into other languages such as C, Java, Etc.
- To provide the scripting capabilities to other languages we can use the python code in it.

4.2.3.2 HTML

Hypertext Markup Language (HTML) is the standard markup language for creating web pages and web applications. With Cascading Style Sheets (CSS) and JavaScript it forms a triad of cornerstone technologies for the World Wide Web. Web browsers receive HTML documents from a web server or from local storage and render them into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

4.2.3.3 Visual Studio Code

Visual Studio Code (VS Code) is a source-code editor developed by Microsoft. It is widely used in computer programming across multiple languages, with strong support for JavaScript, TypeScript, Python, C++, Java, and more. VS Code provides syntax highlighting, intelligent code completion, debugging tools, integrated Git and GitHub support, and a vast marketplace of extensions that allow developers to tailor the editor to their

specific workflows. It supports web development, cloud-native applications, and data science projects through extensions such as Jupyter, Docker, and Azure integrations.

VS Code is cross-platform, available for Windows, macOS, and Linux, and is known for being lightweight yet highly customizable. It is released as proprietary freeware, but its core is based on the open-source project Code - OSS, making it extensible and community-driven.

4.2.3.3.1 Features

- **Intelligent code editing (IntelliSense):** Smart code completion, syntax highlighting, error detection, linting integration, and quick fixes powered by extensions such as Pylance.
- **Project and code navigation:** Explorer view for file structure, global search, quick file navigation, Go to Definition, Find All References, and symbol search across the workspace.
- **Refactoring support:** Rename symbol, extract method, extract variable, organize imports, and other refactoring actions (available via built-in features and extensions).
- **Extensive framework support:** Rich ecosystem of extensions for frameworks such as Django, Flask, and FastAPI, as well as frontend and backend web technologies.
- **Integrated debugging:** Built-in debugger supporting breakpoints, call stack inspection, variable watches, and step-through execution for multiple languages including Python.
- **Integrated testing tools:** Support for unit testing frameworks (e.g., pytest, unittest) with test discovery, execution, and code coverage via extensions.
- **Cloud and remote development:** Extensions for cloud platforms such as Azure, AWS, and Google Cloud, plus remote development via SSH, WSL, and containers.
- **Version control integration:** Built-in Git support with staging, branching, merging, diff views, and source control panel integration; additional support for other systems via extensions.

- **Data science and scientific tools support:** Extensions for Jupyter Notebooks, NumPy, pandas, matplotlib, and interactive Python environments.
- **Extensible and customizable platform:** Large marketplace of extensions for various languages and tools, making it adaptable beyond Python development.
- **Competitive positioning:** Competes with IDEs such as Eclipse (with PyDev), IntelliJ IDEA, and Komodo IDE, while offering a lightweight, highly extensible alternative.

4.2.3.4 ESP32 MICROCONTROLLER

The ESP32 is a low-cost, low-power system on a chip (SoC) microcontroller with integrated Wi-Fi and dual-mode Bluetooth. Developed by Espressif Systems, it is highly integrated with built-in antenna switches, RF balun, power amplifiers, and power management modules. It is engineered specifically for mobile devices, wearable electronics, and IoT (Internet of Things) applications.

Unlike traditional microcontrollers that require external modules to connect to the internet, the ESP32's built-in wireless capabilities make it the perfect "brain" for our fire extinguishing robot, allowing it to control the physical hardware (motors and pumps) while simultaneously transmitting real-time alerts to the Python web dashboard over Wi-Fi.

4.2.3.4. FEATURES

- Microcontroller: Tensilica Xtensa Dual-Core 32-bit LX6 microprocessor.
- Operating Voltage: 3.3V.
- Input Voltage (recommended): 5V (via micro-USB port or VIN pin).
- Digital I/O Pins: 34 (supports PWM, SPI, I2C).
- Analog Input Pins: 18 (12-bit ADC channels).

- Clock Speed: 160 MHz or 240 MHz (making it much faster than standard boards).
- Wireless Connectivity: Integrated 802.11 b/g/n Wi-Fi and Bluetooth v4.2 BR/EDR & BLE.
- DC Current per I/O Pin: 12 mA (recommended).

4.2.3.5 ARDUINO IDE

The Arduino IDE is an open-source software, which is used to write and upload code to the Arduino boards. The IDE application is suitable for different operating systems such as Windows, Mac OS X, and Linux. It supports the programming languages C and C++. Here, IDE stands for Integrated Development Environment. The program or code written in the Arduino IDE is often called as sketching. We need to connect the Genuino and Arduino board with the IDE to upload the sketch written in the Arduino IDE software. The sketch is saved with the extension 'ino'.

The Arduino IDE will appear as:

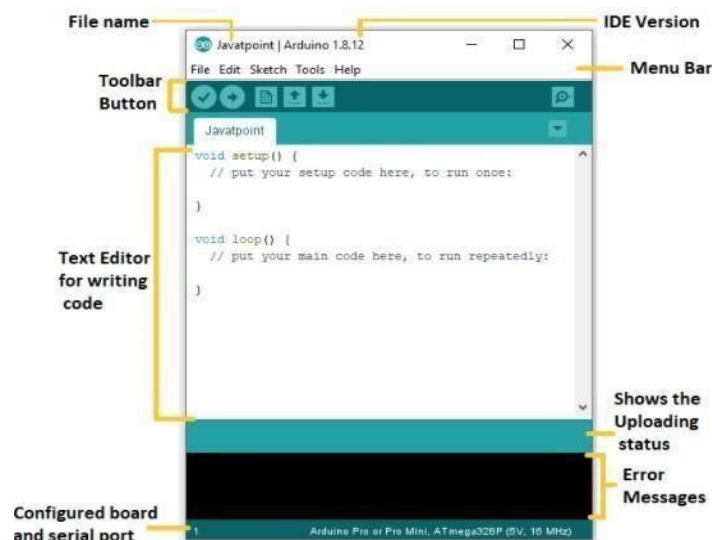


Fig.4.8 Arduino IDE

4.2.3.6 FUNTIONAL REQUIRMENTS

4..2.3.6.1 ESP32 MICROCONTROLLER

The ESP32 is the central processing unit of the robot. It continuously reads data from the IR flame sensors, controls the motor drivers for navigation, and triggers the water pump. Its built-in Wi-Fi module is critical for instantly transmitting real-time fire alerts to the web dashboard without needing an external network module.



Fig.4.9 ESP32 Microcontroller

4.2.3.6.2 IR FLAME SENSOR

The IR flame sensor is the primary detection mechanism used to identify the presence of a fire. It works by detecting the specific infrared radiation wavelengths emitted by flames. When a fire outbreak occurs, it immediately sends a digital signal to the ESP32 to trigger the autonomous suppression system.

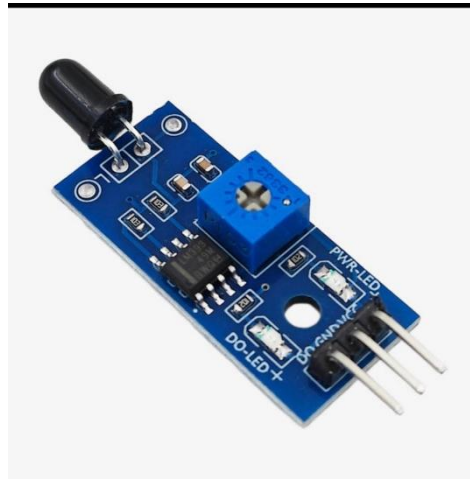


Fig.4.10 IR Flame Sensor

4.2.3.6.3 DC WATER PUMP & RELAY

The DC water pump acts as the primary fire suppression mechanism. Once the robot successfully navigates to the heat source, the ESP32 activates a relay module that powers the pump, forcefully spraying water from the onboard tank to extinguish the flames.



Fig.4.11 Relay



Fig.4.11 DC Water Pump

4.2.4 EXPERIMENTAL SETUP



Fig.4.13 IoT Fire Extinguishing Robot

4.2.5 TABLES USED

Tables

Tables are the fundamental objects in a relational database that store data in a structured format. Each table consists of rows and columns, where rows represent records and columns represent attributes (fields) of the data. A database is essentially a collection of related tables that work together to store and manage information efficiently.

Before storing data, it is important to design tables carefully. This includes defining appropriate data types for each column, setting primary keys to uniquely identify records, establishing foreign keys to maintain relationships between tables, and applying constraints such as NOT NULL, UNIQUE, and CHECK to ensure data integrity. Proper table design reduces redundancy and improves consistency in the database system.

Database Diagrams

Database diagrams provide a graphical representation of database structure. They visually display tables, columns, keys, and relationships between tables. This helps developers and database administrators understand how different tables are connected.

Before working with database diagrams, one must understand how to create a diagram, add database objects to it, modify relationships, and save the diagram. Database diagrams are especially useful during the design phase of a project, as they help in identifying logical errors and improving database structure before implementation.

Indexes

Indexes are database objects used to improve the speed and performance of data retrieval operations. When large amounts of data are stored in tables,

searching without an index can be slow. An index works similarly to an index in a book, allowing the database to locate data quickly without scanning the entire table.

To create efficient indexes, it is important to understand where indexing is required and how it affects performance. While indexes improve SELECT query performance, excessive indexing can slow down INSERT, UPDATE, and DELETE operations. Therefore, indexes must be created and maintained carefully for optimal performance.

Views

A view is a virtual table created based on the result of a SQL query. It does not store data physically but displays data from one or more tables. Views are mainly used to simplify complex queries and enhance security.

By creating and maintaining views, database administrators can control what data users can access. Views help in presenting customized data to different users without giving direct access to the original tables. This ensures better security and logical data abstraction.

Stored Procedures

Stored procedures are precompiled SQL statements stored within the database. They are used to perform specific tasks such as inserting, updating, deleting, or retrieving data.

Using stored procedures improves application performance because the SQL code is compiled once and reused multiple times. They also enhance security by restricting direct access to tables and ensuring that business logic is handled at the database level. Stored procedures simplify application development and reduce repetitive coding.

Triggers

Triggers are special types of stored procedures that automatically execute in response to specific database events such as INSERT, UPDATE, or DELETE operations. They are mainly used to maintain data integrity and enforce business rules.

Triggers can cascade changes to related tables, prevent invalid transactions, enforce complex validation rules, and compare the state of a table before and after a data modification. Based on this comparison, appropriate actions can be taken automatically. This ensures consistency and reliability within the database system.

Register Table

The Register Table is used to store the personal and login information of users who access the smart attendance system. It contains details such as name, email, gender, age, profile image, address, username, password, and confirm password. This table helps in managing user authentication and maintaining user records within the system.

Table: Register

Field Name	Data Type	Size	Constraints	Description
id	Integer	—	Primary Key, Auto Increment	Unique identifier for each user
name	Varchar	255	Null Allowed	Name of the user
email	Varchar	255	Unique	Email address of the user
gender	Varchar	10	Null Allowed	Gender of the user
age	Integer	—	Null Allowed	Age of the user
image	Image/File	—	Null Allowed	Profile image of the user
address	Text	—	Null Allowed	Address of the user
username	Varchar	255	Null Allowed	Username for login
password	Varchar	255	Null Allowed	Password for login
compass	Varchar	255	Null Allowed	Confirm password field

Table 1: Register

5. TESTING

Testing is a crucial phase in the development of the IoT Fire Extinguishing Robot. It ensures that both hardware and software components function correctly and that the complete system performs reliably under real-time conditions. Multiple levels of testing were conducted to evaluate the system's performance, accuracy, stability, and safety. The testing phase helped in identifying errors, improving system response time, and ensuring proper coordination between embedded components and IoT communication modules.

5.1. UNIT TESTING

Unit testing was performed to verify the proper functioning of individual components of the IoT Fire Extinguishing Robot. Each hardware and software module was tested independently before integration to ensure reliability and accuracy.

The flame sensor was tested separately using controlled fire sources to ensure accurate flame detection and sensitivity within a defined range. The water pump were tested individually to confirm proper activation, directional control, and consistent water flow. The ESP32 microcontroller was tested to verify correct GPIO operations, stable Wi-Fi connectivity. The power supply module was also tested to ensure stable voltage regulation and uninterrupted functioning. Each unit was validated under different environmental conditions before combining them into the complete system.

5.2 SYSTEM TESTING

System testing involved evaluating the complete Fire Extinguishing Robot as a fully integrated system. All hardware components including ESP32, flame sensors, motor driver, and water pump were connected and tested together.

During testing, a controlled fire scenario was simulated. The robot successfully detected the flame using the IR sensor, processed the signal through the ESP32, navigated toward the fire source, activated the water

pump, and extinguished the flame effectively. Simultaneously, ensuring remote monitoring capability. The system was tested repeatedly to verify consistency, response time, and reliability under different lighting and environmental conditions. The results confirmed that the entire system worked as expected without delays or malfunction.

5.3 MODULE LEVEL TESTING

Module-level testing focused on verifying each functional module independently to ensure proper performance and compliance with project requirements.

The Fire Detection Module was tested for sensitivity, detection range, and false triggering conditions. The Robot Navigation Module was tested to ensure accurate movement and directional control toward the detected flame. The Water Pump Activation Module was examined to confirm timely and sufficient water discharge. Each module was tested under varying operational conditions to ensure reliability and robustness.

5.4 INTEGRATION TESTING

Integration testing verified that all modules operated together seamlessly without communication errors. After integrating sensors, motors, and communication systems, the system was tested to ensure smooth coordination between hardware and software components.

When the flame sensor detected fire, the signal was processed by the ESP32, which simultaneously controlled robot navigation and activated the extinguishing mechanism. At the same time, This confirmed successful synchronization between detection, movement, extinguishing, and communication modules.

5.5 VALIDATION TESTING

Validation testing ensured that the system met the defined project objectives and stakeholder requirements. The robot was tested in real-time fire simulation scenarios to validate accurate detection, rapid response, effective extinguishing.

The system demonstrated fast response time, reliable operation, and consistent performance. The validation process confirmed that the project fulfills its intended goal of providing an IoT-enabled automated fire safety solution for homes, laboratories, and small industrial environments.

5.6 RECOVERY TESTING

Recovery testing evaluated how the system handled unexpected failures such as Wi-Fi disconnection, sudden power interruption, and sensor malfunction. The robot was intentionally subjected to these failure scenarios to observe system behavior.

After restoring power or network connectivity, the system successfully resumed normal operation. Error-handling mechanisms implemented in the program ensured that the robot did not crash or behave unpredictably. This testing confirmed the system's stability and resilience under adverse conditions.

5.7 OUTPUT TESTING

Output testing verified that the system outputs were accurate, complete, and properly formatted. This included checking sensor readings displayed on the serial monitor, and status updates transmitted through Wi-Fi to the user's device.

All outputs were cross-verified with actual system behavior to ensure correctness. The providing users with accurate real-time updates about fire detection and system status.

5.8 SOURCE CODE TESTING

Source code testing involved reviewing and testing the ESP32 program to detect logical errors, runtime issues, and potential security vulnerabilities. The code was analyzed for proper execution flow, memory optimization, and efficient sensor data processing.

Static code review and multiple test runs were conducted to ensure stable performance. Necessary optimizations were implemented to enhance system speed, reduce latency, and improve reliability. The finalized code ensured smooth integration between hardware components and IoT communication modules.

6. CONCLUSION

6.1 CONCLUSION

The IoT Fire Extinguishing Robot represents an innovative and practical approach to enhancing fire safety using embedded systems and Internet of Things technology. By integrating flame sensors, ESP32 microcontroller, water pump, and wireless communication capabilities, the system can autonomously detect and suppress small-scale fires while simultaneously notifying users in real time.

The project successfully demonstrates how IoT-enabled robotics can significantly reduce response time during fire emergencies. The automated detection and suppression mechanism minimizes human intervention and reduces potential property damage. Overall, the system provides an efficient, cost-effective, and intelligent solution for modern fire safety applications.

6.2 FUTURE WORKS

The system can be further enhanced in several ways to improve its efficiency, scalability, and intelligence. Additional sensors such as smoke and temperature sensors can be incorporated to increase detection accuracy and reduce false alarms. AI-based fire recognition using camera modules can be implemented for advanced image-based detection.

Integration of a GSM module can enable SMS alerts in areas where Wi-Fi connectivity is unavailable. The water storage capacity can be increased for handling larger fire incidents. Obstacle detection using ultrasonic sensors can be added to improve autonomous navigation. A dedicated mobile application can also be developed for real-time monitoring and control.

In the future, this system can be upgraded into a fully autonomous industrial-grade fire safety solution suitable for residential buildings, warehouses, laboratories, and commercial spaces.

7.REFERENCES

7.1. JOURNALS

- S. R. Dixit, et al. "Fire Fighting Robot Using Internet of Things (IoT) for Various Temperature Levels." *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 9, no. 4, 2020, pp. 15-20.
- A. Al-Naimi, et al. "A Smart Firefighting Robot System (LAHEEB)." *IEEE Access*, vol. 8, 2021, pp. 45-53.
- M. A. Kumar, et al. "Multiple Sensors Based Fire Extinguisher Robot Based on DTMF, Bluetooth and GSM Technology." *International Journal of Engineering Trends and Technology*, vol. 18, no. 2, 2019.
- R. Sharma. "Design and Implementation of an Autonomous Fire-Extinguishing Robot with SMS Alert Functionality." *Journal of Robotics and Automation*, vol. 12, no. 1, 2022, pp. 112-118.
- P. Desai, et al. "IoT Enabled Fire Fighting Robot Integrated with ThingSpeak Cloud for Emergency." *Global Journal of Computer Science and Technology*, 2021.

7.2 BOOKS

- Schwartz, Marco. *Internet of Things with ESP32*. Packt Publishing, 2018.
- Monk, Simon. *Programming Arduino: Getting Started with Sketches*. McGraw-Hill Education, 2016.
- Kamal, Raj. *Internet of Things: Architecture and Design Principles*. McGraw-Hill, 2017.
- McRoberts, Michael. *Beginning Arduino*. Apress, 2013.

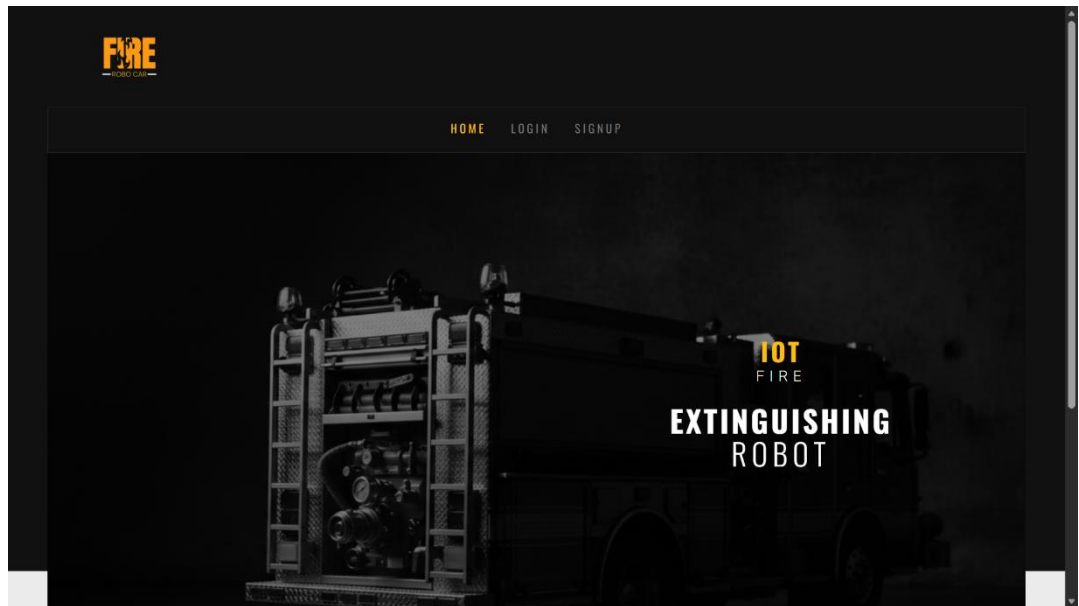
7.3WEBSITES

- Espressif Systems - ESP32 Documentation - <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- Arduino Official Reference - <https://www.arduino.cc/reference/en/>
- ThingSpeak for IoT - MathWorks - <https://thingspeak.com/>
- Python Official Documentation - <https://docs.python.org/3/>

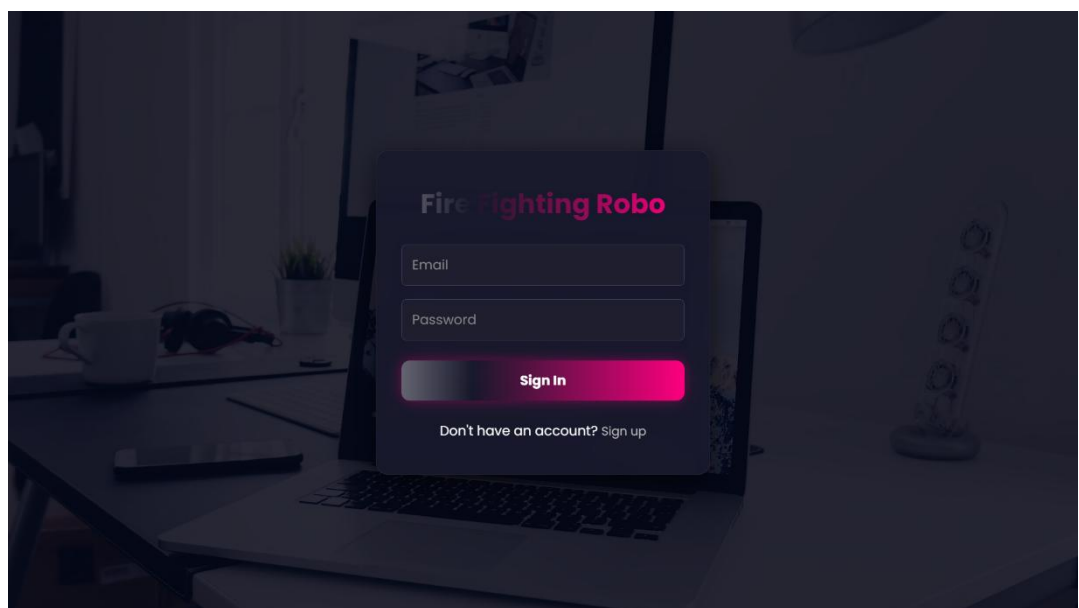
8. APPENDICES

8.1 SCREENSHOTS

HOME



LOGIN



REGISTRATION

Fire Fighting Robo

Full Name

Username

Email

Age

Gender

Address

Choose File

No file chosen

Password

Confirm Password

Sign Up

Already have an account? Sign In

DASHBOARD

EMERGENCY RESPONSE

FIREBOT

100% AUTONOMOUS FIREFIGHTING UNIT v1.0

FLAME

CLEAR

STOPPED

MODE

AUTO

FLAME SENSORS

LEFT

OK

FRONT

OK

RIGHT

OK

AUTO MODE - TAP TO SWITCH MANUAL

DRIVE CONTROLS

STOP

ACTIVATE PUMP

SSID: FireRobot | MAC: 12:34:56:78 | IP: 192.168.4.1

9. SOURCE CODE

9.1 Hardware code

```
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <Arduino.h>

// ===== AP MODE CONFIG =====
const char* ap_ssid    = "FireRobot";
const char* ap_password = "12345678";

AsyncWebServer server(80);

// ===== Pin Definitions =====
#define IN1      21
#define IN2      19
#define IN3      23
#define IN4      22
#define FL_FRONT 39
#define FL_LEFT  34
#define FL_RIGHT 35
#define FIRE_PUMP 15

// ===== Settings =====
#define FLAME_THRESHOLD 80
#define PUMP_TIME      1500

bool autoMode    = true;
String movementStatus = "IDLE";
String flameStatus  = "CLEAR";

// ===== PREMIUM UI =====
const char index_html[] PROGMEM = R"rawliteral(
```

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0,
user-scalable=no">
<title>FireRobot Command</title>
<style>
:root {
  --fire:#ff4500;--fire2:#ff7b00;--ember:#ffb347;
  --bg:#0a0a0c;--surface:#111116;--surface2:#1a1a22;
  --border:#2a2a38;--text:#e8e8f0;--muted:#6a6a82;
  --safe:#00e5a0;
}
* {box-sizing:border-box;margin:0;padding:0;-webkit-tap-highlight-
color:transparent;}
body {background:var(--bg);color:var(--text);font-family:'Courier
New',monospace;min-height:100vh;overflow-x:hidden;}
body::before { content:"";position:fixed;inset:0;
background:repeating-linear-gradient(0deg,transparent,transparent
40px,rgba(255,69,0,.03) 40px,rgba(255,69,0,.03) 41px),
repeating-linear-gradient(90deg,transparent,transparent
40px,rgba(255,69,0,.03) 40px,rgba(255,69,0,.03) 41px);
pointer-events:none;z-index:0;}
.container {position:relative;z-index:1;max-width:480px;margin:0
auto;padding:16px;}
.header {text-align:center;padding:20px 0 16px;}
.header-badge {display:inline-
block;background:rgba(255,69,0,.1);border:1px solid rgba(255,69,0,.3);
color:var(--fire);font-size:10px;letter-spacing:3px;text-
transform:uppercase;
padding:4px 12px;border-radius:2px;margin-bottom:8px;}
.header h1 {font-size:28px;font-weight:700;letter-spacing:-1px;color:var(--
text);}

```

```

.header h1 span{background:linear-gradient(90deg,var(--fire),var(--
fire2),var(--ember));
  -webkit-background-clip:text;-webkit-text-fill-
color:transparent;background-clip:text;}
.header-sub{font-size:11px;color:var(--muted);letter-spacing:2px;margin-
top:4px;}
.status-grid{display:grid;grid-template-columns:1fr 1fr
1fr;gap:8px;margin:16px 0;}
.stat-card{background:var(--surface);border:1px solid var(--border);border-
radius:8px;
  padding:12px 8px;text-align:center;transition:border-color .3s;}
.stat-card.flame-active{border-color:var(--fire);animation:flamePulse .8s
ease-in-out infinite alternate;}
@keyframes flamePulse{from{box-shadow:0 0 8px
rgba(255,69,0,.2);}to{box-shadow:0 0 24px rgba(255,69,0,.5);}}
.stat-label{font-size:9px;letter-spacing:2px;color:var(--muted);text-
transform:uppercase;margin-bottom:6px;}
.stat-value{font-size:12px;font-weight:700;letter-spacing:1px;word-
break:break-word;}
.stat-value.safe{color:var(--safe)} .stat-value.fire{color:var(--fire)}
.stat-value.mode{color:var(--ember)} .stat-value.move{color:var(--text)}
.stat-icon{font-size:20px;margin-bottom:4px;line-height:1;}
.sensor-section{background:var(--surface);border:1px solid var(--
border);border-radius:8px;padding:14px;margin-bottom:10px;}
.sensor-title{font-size:9px;letter-spacing:3px;color:var(--muted);text-
transform:uppercase;margin-bottom:12px;
  display:flex;align-items:center;gap:8px;}
.sensor-title::after{content:"";flex:1;height:1px;background:var(--border);}
.sensor-row{display:flex;align-items:center;gap:10px;margin-bottom:8px;}
.sensor-row:last-child{margin-bottom:0;}
.sensor-name{font-size:11px;color:var(--muted);width:40px;text-
align:right;}
.sensor-bar-wrap{flex:1;height:6px;background:var(--surface2);border-
radius:3px;overflow:hidden;}

```

```

.sensor-bar{height:100%;background:linear-gradient(90deg,var(--
fire),var(--fire2));border-radius:3px;width:0%;transition:width .5s ease;}
.sensor-bar.danger{background:linear-gradient(90deg,#f00,var(--fire));}
.sensor-pct{font-size:11px;color:var(--muted);width:32px;text-align:right;}
.mode-section{margin-bottom:10px;}
.mode-btn{width:100%;padding:13px;border:1px solid var(--
border);border-radius:8px;
background:var(--surface);color:var(--text);font-family:'Courier
New',monospace;
font-size:12px;letter-spacing:2px;text-transform:uppercase;cursor:pointer;
transition:all .2s;display:flex;align-items:center;justify-
content:center;gap:10px;}
.mode-btn:hover{background:var(--surface2);border-color:var(--ember);}
.mode-btn:active{transform:scale(.98);}
.mode-indicator{width:8px;height:8px;border-
radius:50%;background:var(--safe);
box-shadow:0 0 8px var(--safe);animation:blink 1.5s ease-in-out
infinite;}
.mode-indicator.manual{background:var(--ember);box-shadow:0 0 8px
var(--ember);animation:none;}
@keyframes blink{0%,100%{opacity:1;}50%{opacity:.3;}}
.controls-section{background:var(--surface);border:1px solid var(--
border);border-radius:8px;padding:14px;margin-bottom:10px;}
.controls-section.disabled{opacity:.35;pointer-events:none;}
.section-label{font-size:9px;letter-spacing:3px;color:var(--muted);text-
transform:uppercase;
margin-bottom:10px;display:flex;align-items:center;gap:8px;}
.section-label::after{content:"";flex:1;height:1px;background:var(--
border);}
.dpad{display:grid;grid-template-columns:1fr 1fr 1fr;grid-template-
rows:1fr 1fr 1fr;gap:6px;
max-width:200px;margin:0 auto;}
.btn{background:var(--surface2);border:1px solid var(--border);border-
radius:6px;

```

```

    color:var(--text);font-family:'Courier New',monospace;font-
size:18px;cursor:pointer;
    transition:all .15s;aspect-ratio:1;display:flex;align-items:center;justify-
content:center;
    user-select:none;-webkit-user-select:none;}
.btn:hover{background:var(--surface);border-color:var(--fire);color:var(--
fire);}
.btn:active,.btn.pressed{background:rgba(255,69,0,.15);border-color:var(--
fire);
    color:var(--fire);transform:scale(.94);box-shadow:inset 0 0 12px
rgba(255,69,0,.2);}
.btn.stop-btn{background:rgba(255,69,0,.08);border-
color:rgba(255,69,0,.3);font-size:11px;
    color:var(--fire);letter-spacing:1px;}
.btn-empty{visibility:hidden;}
.pump-btn{width:100%;padding:16px;border-radius:8px;border:1px solid
rgba(0,229,160,.3);
    background:rgba(0,229,160,.06);color:var(--safe);font-family:'Courier
New',monospace;
    font-size:13px;letter-spacing:3px;text-transform:uppercase;cursor:pointer;
    transition:all .2s;display:flex;align-items:center;justify-
content:center;gap:10px;
    margin-bottom:10px;position:relative;overflow:hidden;}
.pump-btn::before{content:"";position:absolute;inset:0;
    background:linear-
gradient(90deg,transparent,rgba(0,229,160,.1),transparent);
    transform:translateX(-100%);transition:transform 0s;}
.pump-btn.firing::before{transform:translateX(100%);transition:transform
1.5s ease-in-out;}
.pump-btn:hover{background:rgba(0,229,160,.12);border-color:var(--
safe);}
.pump-btn.firing{background:rgba(0,229,160,.18);box-shadow:0 0 20px
rgba(0,229,160,.2);}
.footer{text-align:center;padding:12px 0 20px;}

```



```

.wifi-badge{display:inline-flex;align-
items:center;gap:8px;background:var(--surface);
border:1px solid var(--border);border-radius:6px;padding:8px 14px;font-
size:11px;color:var(--muted);}
.wifi-dot{width:6px;height:6px;border-radius:50%;background:var(--safe);
box-shadow:0 0 6px var(--safe);animation:blink 2s infinite;}
</style>
</head>
<body>
<div class="container">
  <div class="header">
    <div class="header-badge">&#9888; EMERGENCY RESPONSE</div>
    <h1>FIRE<span>BOT</span></h1>
    <div class="header-sub">AUTONOMOUS FIREFIGHTING UNIT
v1.0</div>
  </div>
  <div class="status-grid">
    <div class="stat-card" id="flameCard">
      <div class="stat-icon">&#128293;</div>
      <div class="stat-label">FLAME</div>
      <div class="stat-value safe" id="flameStatus">CLEAR</div>
    </div>
    <div class="stat-card">
      <div class="stat-icon">&#129302;</div>
      <div class="stat-label">MOTION</div>
      <div class="stat-value move" id="moveStatus">IDLE</div>
    </div>
    <div class="stat-card">
      <div class="stat-icon">&#9881;</div>
      <div class="stat-label">MODE</div>
      <div class="stat-value mode" id="modeStatus">AUTO</div>
    </div>
  </div>
  <div class="sensor-section">

```

```

<div class="sensor-title">FLAME SENSORS</div>
<div class="sensor-row">
  <div class="sensor-name">LEFT</div>
  <div class="sensor-bar-wrap"><div class="sensor-bar"
id="barLeft"></div></div>
  <div class="sensor-pct" id="pctLeft">--%</div>
</div>
<div class="sensor-row">
  <div class="sensor-name">FWD</div>
  <div class="sensor-bar-wrap"><div class="sensor-bar"
id="barFront"></div></div>
  <div class="sensor-pct" id="pctFront">--%</div>
</div>
<div class="sensor-row">
  <div class="sensor-name">RIGHT</div>
  <div class="sensor-bar-wrap"><div class="sensor-bar"
id="barRight"></div></div>
  <div class="sensor-pct" id="pctRight">--%</div>
</div>
</div>
<div class="mode-section">
  <button class="mode-btn" onclick="toggleMode()" id="modeBtn">
    <div class="mode-indicator" id="modeIndicator"></div>
    <span id="modeBtnText">AUTO MODE &mdash; TAP TO
SWITCH</span>
  </button>
</div>
<div class="controls-section disabled" id="controlPad">
  <div class="section-label">DRIVE CONTROLS</div>
  <div class="dpad">
    <div class="btn-empty"></div>
    <div class="btn" id="btnFwd" onpointerdown="cmd('forward')"
onpointerup="cmd('stop')" onpointerleave="cmd('stop')">&#9650;</div>
    <div class="btn-empty"></div>

```

```

    <div class="btn" id="btnLeft" onpointerdown="cmd('left'"
onpointerup="cmd('stop')" onpointerleave="cmd('stop')">#9664;</div>
    <div class="btn stop-btn" onclick="cmd('stop')">STOP</div>
    <div class="btn" id="btnRight" onpointerdown="cmd('right'"
onpointerup="cmd('stop')" onpointerleave="cmd('stop')">#9654;</div>
    <div class="btn-empty"></div>
    <div class="btn" id="btnBack" onpointerdown="cmd('back')'"
onpointerup="cmd('stop')" onpointerleave="cmd('stop')">#9660;</div>
    <div class="btn-empty"></div>
</div>
</div>
<button class="pump-btn" id="pumpBtn" onclick="firePump()">
    <span>#128167;</span>
    <span id="pumpText">ACTIVATE PUMP</span>
</button>
<div class="footer">
    <div class="wifi-badge">
        <div class="wifi-dot"></div>
        SSID: FireRobot &nbsp;|&nbsp; PWD: 12345678 &nbsp;|&nbsp;
192.168.4.1
    </div>
</div>
</div>
<script>
    let isAuto=true;
    function cmd(v){
        fetch('/cmd?val='+v).catch(()=>{});

document.querySelectorAll('.btn').forEach(b=>b.classList.remove('pressed')
);
    const m={forward:'btnFwd',back:'btnBack',left:'btnLeft',right:'btnRight'};
    if(m[v])document.getElementById(m[v])?.classList.add('pressed');
    }

```

```

function
toggleMode(){fetch('/mode').then(()=>updateStatus()).catch(()=>{ });}

function firePump(){
  const btn=document.getElementById('pumpBtn');
  const txt=document.getElementById('pumpText');
  btn.classList.add('firing');txt.textContent='PUMPING...';
  fetch('/cmd?val=pump').catch(()=>{ });

  setTimeout(()=>{btn.classList.remove('firing');txt.textContent='ACTIVATE
PUMP';},1600);
}

function updateStatus(){
  fetch('/status').then(r=>r.json()).then(d=>{
    const hasFlame=d.flame&& d.flame.includes('Flame');
    const fs=document.getElementById('flameStatus');
    fs.textContent=hasFlame?d.flame.replace(/[\^\w\s]/g,"").trim():'CLEAR';
    fs.className='stat-value '+ (hasFlame?'fire':'safe');
    document.getElementById('flameCard').className='stat-card
'+(hasFlame?'flame-active:');
    document.getElementById('moveStatus').textContent=d.move||'IDLE';
    isAuto=d.mode==='AUTO';
    document.getElementById('modeStatus').textContent=d.mode;
    document.getElementById('modeBtnText').textContent=
      isAuto?'AUTO MODE \u2014 TAP TO SWITCH
MANUAL':'MANUAL MODE \u2014 TAP TO SWITCH AUTO';
    document.getElementById('modeIndicator').className='mode-
indicator'+(isAuto?'':' manual');
    document.getElementById('controlPad').className='controls-
section'+(isAuto?' disabled:');
  }).catch(()=>{ });
}

function updateSensors(){
  fetch('/sensors').then(r=>r.json()).then(d=>{
    const set=(bar,pct,v)=>{

```

```

        document.getElementById(bar).style.width=v+'%';
        document.getElementById(bar).className='sensor-bar'+(v>80?'
danger:');
        document.getElementById(pct).textContent=v+'%';
    };
    set('barLeft','pctLeft',d.left||0);
    set('barFront','pctFront',d.front||0);
    set('barRight','pctRight',d.right||0);
    }).catch(()=>{});
}
setInterval(updateStatus,800);
setInterval(updateSensors,600);
updateStatus();
</script>
</body>
</html>
)rawliteral";

```

```

// ===== Motor Control =====

void moveForward() { movementStatus="FORWARD";
digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH); digitalWrite(IN3,LOW);
digitalWrite(IN4,HIGH); }
void moveBackward() { movementStatus="BACKWARD";
digitalWrite(IN1,HIGH);digitalWrite(IN2,LOW);
digitalWrite(IN3,HIGH);digitalWrite(IN4,LOW); }
void turnLeft() { movementStatus="LEFT";
digitalWrite(IN1,HIGH);digitalWrite(IN2,LOW); digitalWrite(IN3,LOW);
digitalWrite(IN4,HIGH); }
void turnRight() { movementStatus="RIGHT"; digitalWrite(IN1,LOW);
digitalWrite(IN2,HIGH); digitalWrite(IN3,HIGH);digitalWrite(IN4,LOW);
}
void stopMotors() { movementStatus="STOPPED";
digitalWrite(IN1,LOW); digitalWrite(IN2,LOW); digitalWrite(IN3,LOW);
digitalWrite(IN4,LOW); }

```

```

void activatePump() { digitalWrite(FIRE_PUMP,HIGH);
delay(PUMP_TIME); digitalWrite(FIRE_PUMP,LOW); }

// ===== Auto Flame Logic =====
void autoFire() {
  int f = map(analogRead(FL_FRONT), 4095, 0, 0, 100);
  int l = map(analogRead(FL_LEFT), 4095, 0, 0, 100);
  int r = map(analogRead(FL_RIGHT), 4095, 0, 0, 100);

  if (f > FLAME_THRESHOLD) { flameStatus = "Front Flame";
moveForward(); delay(300); stopMotors(); activatePump(); }
  else if (l > FLAME_THRESHOLD) { flameStatus = "Left Flame";
turnLeft(); delay(300); stopMotors(); activatePump(); }
  else if (r > FLAME_THRESHOLD) { flameStatus = "Right Flame";
turnRight(); delay(300); stopMotors(); activatePump(); }
  else { flameStatus = "CLEAR"; stopMotors(); }
}

// ===== Setup =====
void setup() {
  Serial.begin(115200);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(FIRE_PUMP, OUTPUT);

  WiFi.softAP(ap_ssid, ap_password);
  Serial.print("AP IP: "); Serial.println(WiFi.softAPIP());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *req){
    req->send_P(200, "text/html", index_html);
  });

  server.on("/mode", HTTP_GET, [](AsyncWebServerRequest *req){
    autoMode = !autoMode;

```

```

    req->send(200, "text/plain", "OK");
});

```

```

server.on("/cmd", HTTP_GET, [](AsyncWebServerRequest *req){
    if (!autoMode && req->hasParam("val")) {
        String v = req->getParam("val")->value();
        if (v == "forward") moveForward();
        else if (v == "back") moveBackward();
        else if (v == "left") turnLeft();
        else if (v == "right") turnRight();
        else if (v == "stop") stopMotors();
        else if (v == "pump") activatePump();
    }
    req->send(200, "text/plain", "OK");
});

```

```

server.on("/status", HTTP_GET, [](AsyncWebServerRequest *req){
    String json = "{\"flame\":\"" + flameStatus + "\",\"
                  \"move\":\"" + movementStatus + "\",\"
                  \"mode\":\"" + String(autoMode ? "AUTO" : "MANUAL") +
    "\"}\"";
    req->send(200, "application/json", json);
});

```

// NEW: real-time sensor values for the UI progress bars

```

server.on("/sensors", HTTP_GET, [](AsyncWebServerRequest *req){
    int f = map(analogRead(FL_FRONT), 4095, 0, 0, 100);
    int l = map(analogRead(FL_LEFT), 4095, 0, 0, 100);
    int r = map(analogRead(FL_RIGHT), 4095, 0, 0, 100);
    String json = "{\"front\":\"" + String(f) +
                  "\",\"left\":\"" + String(l) +
                  "\",\"right\":\"" + String(r) + "\"}";
    req->send(200, "application/json", json);
});

```

```

server.begin();
}

// ===== Loop =====
void loop() {
  if (autoMode) autoFire();
}

```

9.2 models.py

```

from django.db import models

# Create your models here.
class Register(models.Model):
    name=models.CharField(max_length=255,blank=True,null=True)
    email=models.CharField(max_length=255,unique=True)
    gender=models.CharField(max_length=10,null=True,blank=True)
    age=models.IntegerField(null=True,blank=True)
    image=models.FileField(upload_to="profile/",null=True,blank=True)
    address=models.TextField(null=True,blank=True)
    username=models.CharField(max_length=255,blank=True,null=True)
    password=models.CharField(max_length=255,blank=True,null=True)
    conpass=models.CharField(max_length=255,blank=True,null=True)

```

9.3 views.py

```

from django.shortcuts import render,redirect
from django.http import HttpResponse
from . import models

# Create your views here.

```



```

def index(request):
    return render(request,'index.html')

def login(request):

    if request.method=='POST':
        email=request.POST.get('mail')
        password=request.POST.get('pass')
        try:

            user=models.Register.objects.get(email=email)
            request.session['email']=email
            if user.password==password:
                return redirect('main')
            return HttpResponseRedirect('invalid password')
        except models.Register.DoesNotExist:
            return HttpResponseRedirect('user does not exist')
    return render(request,'login.html')

def main(request):
    return redirect('http://192.168.4.1/')

def signup(request):
    if request.method=='POST':
        name=request.POST.get('name')
        email=request.POST.get('mail')
        gender=request.POST.get('gender')
        adr=request.POST.get('address')
        age=request.POST.get('age')
        img=request.FILES.get('image')
        username=request.POST.get('username')
        password=request.POST.get('pass')
        c_pass=request.POST.get('confirmpass')

```

```

if password==c_pass:
    if models.Register.objects.filter(email=email).exists():
        return HttpResponse("Email already exists")
    else:
        models.Register(name=name,email=email,username=username,p a
ssword=password,conpass=c_pass,gender=gender,address=adr,age=age,ima
ge=img).save()
        return redirect('login')
    return HttpResponse('passwords do not match')
return render(request,'register.html')

```